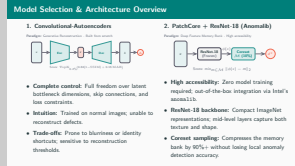


Industrial Component Anomaly Detection

Models

Model Selection & Architecture Overview



Notation on PatchCore diagram ($\phi(x)$ and $m \in \mathcal{M}$):

$\phi(x)$ is the *patch feature representation* extracted from frozen ResNet-18 (concatenated mid-level layers 2+3). $\mathcal{M}$ is the *coreset memory bank* of normal reference vectors, and $m \in \mathcal{M}$ is an individual stored normal patch prototype. $\min_{m \in \mathcal{M}} \|\phi(x) - m\|_2$ is the nearest-neighbour L_2 distance: an anomaly is detected when a query patch vector has no close normal counterpart in memory.

Q: Why two such different paradigms rather than one?

They fail in completely different ways. A CAE can fail silently on textures it has never seen, while PatchCore fails when the memory bank has coverage gaps. Running both gives orthogonal error signals. PatchCore is also a natural upper-bound baseline: if PatchCore cannot detect a defect with frozen ImageNet features, the defect has near-zero visual contrast — no learned model will either.

Q: Couldn't you just use a classifier with defect labels?

Hard no in industrial production: defect labels require defective parts to exist, which means letting defective parts accumulate before deployment. The whole value proposition is Day-1 deployment with zero defect data. Any supervised approach also fails on *novel* defect types not seen during training.

Conceptual trap: PatchCore “trains” vs. “fits”.

PatchCore has no gradient updates. The “fitting” step is just a forward pass through ResNet-18 + writing vectors to memory. The distinction matters: there is no overfitting risk, no learning rate to tune, no epochs. It is closer to database indexing than training.

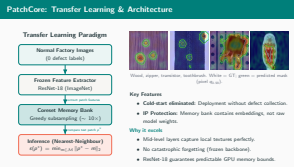
Connection between models: Both ultimately compute a per-patch distance to normality — CAE in pixel reconstruction space,

PatchCore in feature embedding space. The difference is *where* normal is represented: CAE encodes it in weights. PatchCore in

Industrial Component Anomaly Detection

Models

PatchCore: Transfer Learning & Architecture



Inference formula ($s(p^*) = \min_{m \in \mathcal{M}} \|p^* - m\|_2$):

$p^* \equiv \phi(x)_i$ is the query patch feature vector extracted by frozen ResNet-18 $\phi(x)$. $\mathcal{M}$ is the coreset-reduced memory bank of normal training vectors, and $m \in \mathcal{M}$ is the nearest normal reference vector. Scoring is non-parametric nearest-neighbour distance: if a query patch has no close match in $\mathcal{M}$, its distance is large $\Rightarrow$ anomalous.

Q: Why not fine-tune ResNet on the normal images?

Without defect labels you can only do self-supervised fine-tuning. But this risks *destroying* the mid-level texture representations that make PatchCore work — fine-tuning on a narrow distribution (one factory category) collapses the embedding space to only what it has seen, killing generalisation to subtle defects. The frozen features generalise better precisely because they were trained on 1M diverse images.

Q: Why layer2+layer3 of ResNet-18 and not the final layer?

The final layer encodes ImageNet class identity (cat, truck, etc.) — totally irrelevant to surface defects. Layer2/3 encode mid-level statistics: edges, frequency responses, local texture gradients. These are universal and transfer well to any visual texture domain.

Q: How does coreset subsampling work without losing coverage?

Greedy farthest-point sampling: iteratively pick the point in feature space that is farthest from all already-selected points. This guarantees maximum coverage of the feature space manifold with the fewest points. It is *not* random subsampling. Random subsampling at 10% would cluster around dense regions, leaving sparse regions (rare but valid normal appearances) uncovered — producing false positives.

Failure mode to know: PatchCore cannot detect defects that are *globally* anomalous but locally normal. Example: a correct screw

Industrial Component Anomaly Detection

Models

Keras CAE – Autoencoder Learning Process

Keras CAE – Autoencoder Learning Process	
The Concept	Key Design Decisions
Training Phase: - Trained <i>only</i> on defect-free images. - Learns the structural grammar of “normal” surfaces.	Masked Image Modeling (MIM): 25% of patches randomly masked before encoding. Forces the model to understand global context, prevents “identity mapping”.
Inference Phase: - On defective images, anomalies are reconstructed as “normal” parts. - Reconstruction fails at the defect sites $\Rightarrow$ high reconstruction error. - High reconstruction error $\Rightarrow$ Anomaly Signal.	SSIM + MSE Loss: $\mathcal{L} = \alpha(1 - \text{SSIM}) + (1 - \alpha) \text{MSE}$. MSE captures global variance, while SSIM enforces high structural and perceptual fidelity. Architectural Choices: ELU no dying neurons + zero-centered mean. BatchNorm stabilizes covariate shifts. AdamW enables decoupled weight decay.

Q: Isn't the loss the anomaly score? (Classic confusion)

No. The loss is a global image scalar used only during training for weight updates. The anomaly score is a *per-pixel* error map aggregated via Top-K pooling, computed only at inference with frozen weights. They share the MSE signal but serve completely different purposes. The loss cannot localise defects; the pixel error map can.

Q: Why does a decoder trained on normals fail on defects? It still has all its weights.

The bottleneck is a learned *normal manifold*. A defective image gets projected onto the closest point on that manifold, which is a normal surface. The decoder then faithfully decodes *that projection*, not the original defective input. Think of it as a lossy codec trained only on one class: it always outputs something class-like.

Q: Why SSIM and not just MSE?

MSE optimises average pixel intensity independently — a blurry reconstruction can have low MSE. SSIM penalises structural differences: contrast, luminance, and local co-movement of pixels. A blur that preserves colour but destroys edges is heavily penalised by SSIM, not by MSE. The combination forces the reconstruction to be both globally correct (MSE) and structurally faithful (SSIM), which matters for texture surfaces like leather or wood.

Q: Why not skip connections like a U-Net?

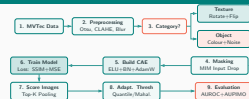
Skip connections let the encoder pass high-frequency detail directly to the decoder, bypassing the bottleneck. This makes it *trivially easy* to reconstruct any input, including defects. The bottleneck *must* be the information bottleneck. MIM reinforces this: if you mask 25% of the input, skip connections cannot carry the masked content, forcing the bottleneck to infer it from context.

Industrial Component Anomaly Detection

Models

Keras CAE – End-to-End Pipeline

Keras CAE – End-to-End Pipeline



Automated Pipeline & Optuna Integration

The pipeline is dynamically orchestrated. Hyperparameters (Preprocessing/Model) (e.g., augmentation strategy, latent dimensions) optimized using Optuna targeting Pixel AUPIMO.

Q: Why optimise Optuna against AUPIMO and not AUROC?

AUROC is threshold-free and integrates over the entire FPR range $[0, 1]$. In industrial settings, operating at $\text{FPR} > 10^{-4}$ is practically useless (too many false alarms). AUPIMO evaluates *only* the operationally relevant region and at pixel level, making it a far tighter proxy for real-world utility. Optimising AUROC could improve the broad curve while degrading the narrow high-stakes region that actually matters.

Q: Why does Optuna target the validation split, not the test set?

Any hyperparameter search against the test set is test leakage *by design*. The 15% normal validation partition (never anomalous, never test) is the only legal signal available for tuning. The test set is locked until final scoring. This is the “zero-leakage” guarantee *within* the training process, not just a one-time split.

Q: What does Optuna actually tune and how?

Bayesian optimisation (Tree-structured Parzen Estimator) over the joint space of: crop size/stride, mask ratio, latent dimension, CLAHE clip limit, augmentation choice. It builds a probabilistic model of which configurations lead to high AUPIMO and samples the next trial from the most promising region. Far more efficient than grid/random search in high-dimensional spaces.

Q: Why does the threshold calibration use the 95th percentile?

It is a 5% false-alarm budget on normal parts ($\alpha = 0.05$). This is the T2 “Precision Industrial” profile. 95% of normal parts score below this threshold; 5% are false positives. Setting α is the only available lever to control the false-alarm rate without defect labels. The key insight: this is a *day-1 deployable SLA guarantee* on the false-alarm rate, even with zero knowledge of what